

Software Engineering: Principles & Patterns Reference

1. S.O.L.I.D. Principles

Principle	Concept	Objective
S - Single Responsibility	A class should have one, and only one, reason to change.	High cohesion; reduces complexity and side-effects.
O - Open/Closed	Entities should be open for extension, but closed for modification.	Maintain existing code stability while adding new features.
L - Liskov Substitution	Subclasses must be substitutable for their base classes without altering correctness.	Ensure inheritance is used correctly and predictably.
I - Interface Segregation	Many client-specific interfaces are better than one general-purpose interface.	Decouple systems; prevent clients from depending on methods they do not use.
D - Dependency Inversion	Depend upon abstractions (interfaces), not concretions (implementations).	Reduce coupling; high-level modules shouldn't depend on low-level modules.

2. Object-Oriented Design Patterns (GoF)

Category	Pattern	Description
Creational (Object creation)	Singleton	Ensures a class has only one instance and provides a global point of access to it.
	Factory Method	Defines an interface for creating an object, but lets subclasses decide which class to instantiate.
	Builder	Separates the construction of a complex object from its representation, allowing the same process to create different representations.

	Abstract Factory	Provides an interface for creating families of related or dependent objects without specifying their concrete classes.
Structural (Object composition)	Adapter	Converts the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise.
	Decorator	Attaches additional responsibilities to an object dynamically, providing a flexible alternative to subclassing for extending functionality.
	Facade	Provides a unified, simplified interface to a set of interfaces in a subsystem, making the subsystem easier to use.
	Proxy	Provides a surrogate or placeholder for another object to control access to it (e.g., lazy loading, access control).
Behavioral (Object interaction)	Observer	Defines a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.
	Strategy	Defines a family of algorithms, encapsulates each one, and makes them interchangeable within the client context.
	Command	Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests.
	State	Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

3. Clean Code & Architectural Best Practices

- **DRY (Don't Repeat Yourself):** Every piece of knowledge must have a single, unambiguous, authoritative representation within a system. Reduces maintenance overhead.
- **KISS (Keep It Simple, Stupid):** Systems work best if they are kept simple rather than made complicated. Avoid unnecessary complexity and over-engineering.
- **YAGNI (You Aren't Gonna Need It):** Always implement things when you actually need them, never when you just foresee that you need them. Prevents code bloat.

- **Composition over Inheritance:** Achieve polymorphic behavior and code reuse by composing objects containing desired functionality rather than inheriting from a base class.
- **Fail Fast:** Systems should immediately report any failure or condition that is likely to lead to failure, rather than attempting to proceed and failing later in unpredictable ways.
- **Law of Demeter (Principle of Least Knowledge):** A given object should assume as little as possible about the structure or properties of anything else (don't talk to strangers).